

Лабораторная работа №28. Веб-сервер на C#: список любимых игр с полным управлением

Цель работы:

- Повторить создание веб-сервера на ASP.NET Core;
- Закрепить понятия маршрутизации и HTTP-методов;
- Научиться хранить данные в памяти сервера;
- Освоить новые HTTP-методы: POST, DELETE и PUT;
- Научиться возвращать правильные HTTP-статусы (200, 201, 404);
- Познакомиться с инструментом curl для отправки HTTP-запросов из терминала.

Краткая теория (вводная часть)

В прошлой работе мы писали все маршруты прямо в **Program.cs**. Это удобно для маленьких проектов, но когда маршрутов становится 20–30, файл превращается в кашу. Контроллеры решают эту проблему: каждая группа маршрутов живёт в отдельном классе. Сервер игр — в **GamesController**, сервер пользователей — в **UsersController** и т.д.

Повторение: что такое HTTP-метод?

В лабораторной работе №27 мы создали веб-сервер и разобрали, что браузер общается с сервером через HTTP-запросы. Каждый запрос имеет **метод** — он говорит серверу, что именно хочет клиент:

Метод	Что означает	Пример
GET	Получить данные	Открыть список игр
POST	Отправить новые данные	Добавить игру в список
DELETE	Удалить данные	Удалить игру из списка

В браузере вы можете отправить только **GET** — просто вбив адрес в строку. Для **POST** и **DELETE** нужен специальный инструмент. Сегодня мы познакомимся с **curl**.

Что такое curl?

curl — программа командной строки для отправки HTTP-запросов. Она уже установлена в Windows 10/11, macOS и Linux — ничего скачивать не нужно.

Проверьте, что curl доступен — введите в терминале:

curl --version

```
└─$ curl --version
curl 8.18.0 (x86_64-pc-linux-gnu)
```

Вы увидите версию программы. Это значит curl работает.

Базовый синтаксис curl:

```
# GET-запрос (по умолчанию)
curl http://localhost:5000/games

# POST-запрос с JSON-телом
curl -X POST http://localhost:5000/games \
  -H "Content-Type: application/json" \
  -d "{\"title\": \"Hollow Knight\"}"

# DELETE-запрос
curl -X DELETE http://localhost:5000/games/1
```

Разберём флаги:

Флаг	Что означает
-X POST	Указать метод запроса
-H "..."	Добавить заголовок (header)
-d "..."	Тело запроса (data)

Что такое HTTP-статус?

Сервер всегда отвечает не только данными, но и **статусным кодом** — числом, которое говорит об успехе или ошибке:

Код	Название	Когда используется
200 OK	Успех	Данные найдены и возвращены
201 Created	Создано	Новый объект успешно добавлен
404 Not Found	Не найдено	Объект с таким id не существует
400 Bad Request	Плохой запрос	Клиент прислал некорректные данные

Сегодня мы будем явно возвращать эти коды из нашего контроллера.

Шаг 1. Создание проекта

1.1. Создание рабочей директории

1. Откройте любой терминал (например **Git Bash**).
2. Перейдите в каталог с документами.
3. Создайте директорию для лабораторной работы и папку для скриншотов:

```
mkdir Lab28_GamesList
```

cd Lab28_GamesList

mkdir img

1.2. Создание проекта Web API

1. На этот раз используем шаблон **webapi** — он создаёт проект с поддержкой контроллеров:

dotnet new webapi -n GamesApi

2. Откройте проект в VS Code:

code .

3. Откройте терминал и выполните:

cd GamesApi

dotnet restore

Отличие от предыдущей работы: шаблон **web** создавал минимальный проект (Minimal API). Шаблон **webapi** создаёт проект с контроллерами, **Swagger** и всем необходимым для реального API.

Изучите созданную структуру:

GamesApi/

|— **Properties/**

| |— **launchSettings.json** ← настройки запуска (порты и т.д.)

|— **Program.cs** ← точка входа

|— **GamesApi.csproj** ← конфигурация проекта

|— **appsettings.json** ← настройки приложения

1.3. Добавление .editorconfig

1. В корне папки **Lab28_GamesList** создайте файл **.editorconfig**

2. Добавьте в него следующие правила (все скобки на той же строке):

```
[*.cs]
csharp_new_line_before_open_brace = none
```

1.4. Инициализация Git

1. Убедитесь, что вы находитесь в папке **Lab28_GamesList** (не внутри **GamesApi**), и выполните в терминале:

cd ..

git init -b main

cd GamesApi

dotnet new gitignore

2. Продолжаем работу в терминале:

cd ..

touch README.md

git add .

git commit -m "Initial commit: GamesApi project"

3. Создайте пустой public-репозиторий на GitHub: **Lab28_GamesList**

4. Привяжите и отправьте:

git remote add origin <URL_репозитория>

git push -u origin main

5. Сделайте скриншот терминала с результатом **push** и сохраните в `img/gitPushLab28_FIO.png`

Шаг 2. Модель данных

Прежде чем писать контроллер, нам нужна **модель** — класс C#, который описывает одну игру в нашем списке.

1. Создайте папку **Models** в **GamesApi** и файл **Models/Game.cs**

2. Откройте **Models/Game.cs** в VS Code и вручную перепишите:

```
namespace GamesApi.Models;
public class Game {
    public int Id { get; set; }
    public string Title { get; set; } = string.Empty;
    public string Genre { get; set; } = string.Empty;
    public int ReleaseYear { get; set; }
}
```

Разберём каждую строку:

Строка	Что означает
namespace GamesApi.Models	Пространство имён — «адрес» класса в проекте
public class Game	Объявление класса
public int Id	Целочисленный идентификатор
public string Title	Название игры
= string.Empty	Значение по умолчанию — пустая строка, не null
public int ReleaseYear	Год выхода

Зачем `= string.Empty`?

В C# строка может быть null — отсутствовать совсем. Это часто вызывает ошибки. Присваивая **string.Empty**, мы гарантируем, что поле всегда содержит хотя бы пустую строку.

Шаг 3. Хранилище данных в памяти

В этой лабораторной мы не используем базу данных — она появится позже. Данные будем хранить **в памяти сервера** — в обычном списке C#.

1. Создайте папку **Data** в **GamesApi** и файл **Data/GamesStore.cs**
2. Откройте **Data/GamesStore.cs** и вручную перепишите:

```
using GamesApi.Models;
namespace GamesApi.Data;

public static class GamesStore {
    private static int _nextId = 4;
    public static List<Game> Games { get; } = new() {
        new Game {
            Id = 1,
            Title = "The Elder Scrolls IV: Oblivion",
            Genre = "RPG",
            ReleaseYear = 2006,
        },
        new Game {
            Id = 2,
            Title = "The Witcher 3: Wild Hunt",
            Genre = "RPG",
            ReleaseYear = 2015,
        },
        new Game {
            Id = 3,
            Title = "Grand Theft Auto: San Andreas",
            Genre = "Action-adventure",
            ReleaseYear = 2004,
        },
    };
    public static int NextId() => _nextId++;
}
```

Можете добавить ещё несколько своих примеров.

Разберём ключевые моменты:

- **public static class GamesStore** — статический класс. Это значит, что он существует в одном экземпляре на всё время работы сервера. Все контроллеры работают с одним и тем же списком.
- **private static int _nextId = 4** — счётчик начинается с 4, потому что у нас уже есть игры с Id 1, 2, 3. Каждый раз когда добавляется новая игра — счётчик увеличивается на 1.
- **public static int NextId() => _nextId++** — метод возвращает текущее значение счётчика и сразу увеличивает его на 1.

Важно: данные хранятся только пока сервер работает. При перезапуске список сбросится к начальным трём играм. Постоянное хранение данных — тема следующих лабораторных.

Кроме того, если два запроса придут одновременно, они могут повредить список — это ещё одна причина, почему в реальных проектах используют базы данных.

Шаг 4. Контроллер

Теперь создадим контроллер — класс, который обрабатывает HTTP-запросы и возвращает ответы.

Создайте папку **Controllers** в **GamesApi** и файл **Controllers/GamesController.cs**

4.1. Заголовок файла — подключение зависимостей

Откройте **Controllers/GamesController.cs** и вручную перепишите:

```
using GamesApi.Models;
using GamesApi.Data;
using Microsoft.AspNetCore.Mvc;
```

Эти три строки подключают всё необходимое для работы контроллера:

- **using GamesApi.Models** — подключаем наш класс **Game** из папки **Models**
- **using GamesApi.Data** — подключаем **GamesStore** — наш список игр из папки **Data**
- **using Microsoft.AspNetCore.Mvc** — подключаем библиотеку ASP.NET Core, которая даёт нам контроллеры, атрибуты маршрутизации ([HttpGet], [HttpPost] и т.д.) и методы ответа (Ok(), NotFound() и т.д.)

```
namespace GamesApi.Controllers;
```

Пространство имён — «адрес» этого файла в проекте. По соглашению **Controllers** лежат в пространстве имён **НазваниеПроекта.Controllers**.

4.2. Объявление класса

```
[ApiController]
[Route("api/[controller]")]
public class GameController : ControllerBase { }
```

[ApiController] — атрибут (аннотация над классом). Атрибуты в C# — это аннотации, которые добавляют поведение классу или методу. Говорит ASP.NET Core: «это API-контроллер». Включает удобные автоматические поведения:

- если клиент прислал некорректные данные — автоматически вернётся 400 Bad Request
- параметры из тела запроса подхватываются автоматически

[Route("api/[controller]")] — задаёт базовый маршрут для всех методов этого класса. **[controller]** — это placeholder, который ASP.NET Core заменяет на имя класса без слова Controller. То есть GameController → Games, и итоговый маршрут будет api/games.

public class GameController : ControllerBase — объявляем класс, который наследуется от ControllerBase. Это базовый класс ASP.NET Core для API-контроллеров. Он даёт нам готовые методы: Ok(), NotFound(), CreatedAtAction() и другие — чтобы не писать HTTP-ответы вручную.

Отличие от обычного класса C#: обычный класс вы вызываете сами через new. Контроллер вызывает фреймворк автоматически, когда приходит HTTP-запрос на нужный маршрут.

Далее пишем внутри класса

4.3. Метод 1 — GET /api/games (получить все игры)

```
[HttpGet]
public ActionResult<List<Game>> GetAll() {
    return Ok(GamesStore.Games);
}
```

[HttpGet] — атрибут, который говорит: этот метод срабатывает на GET-запрос. Маршрут берётся из класса: api/games.

ActionResult<List<Game>> — тип возвращаемого значения. Здесь два слоя:

- **List<Game>** — мы хотим вернуть список игр
- **ActionResult<...>** — обёртка, которая позволяет вернуть либо данные, либо HTTP-статус (например 404). Без неё мы не могли бы вернуть NotFound().

return Ok(GamesStore.Games) — метод Ok() мы получили от ControllerBase. Он формирует ответ: статус 200 OK + данные внутри. GamesStore.Games — это наш список из памяти сервера.

4.4. Метод 2 — GET /api/games/{id} (получить одну игру)

```
[HttpGet("{id}")]
public ActionResult<Game> GetById(int id) {
    var game = GamesStore.Games.FirstOrDefault(g => g.Id == id);
    if (game is null) {
        return NotFound(new { message = $"Игра с id={id} не найдена" });
    }
    return Ok(game);
}
```

[HttpGet("{id}")] — маршрут расширяется до api/games/{id}. Фигурные скобки означают: здесь будет переменная часть URL. Например: api/games/3.

public ActionResult<Game> GetById(int id) — параметр int id ASP.NET Core подставляет автоматически из URL. Если запрос пришёл на api/games/3, то id будет равен 3.

GamesStore.Games.FirstOrDefault(g => g.Id == id) — ищем в списке первую игру, у которой Id совпадает с нашим. g => g.Id == id — это лямбда-выражение, которое вы уже встречали. Если ничего не найдено — возвращает null.

if (game is null) — проверяем, нашлась ли игра. is null — это современная проверка на null в C#, работает так же как == null.

return NotFound(new { message = \$"..." }) — если игра не найдена, возвращаем статус 404 Not Found. Внутри передаём анонимный объект с полем message — это текст ошибки, который увидит клиент. new { message = "..." } — анонимный объект, класс для него создавать не нужно, это просто способ быстро передать данные.

return Ok(game) — если игра найдена, возвращаем её с кодом 200.

4.5. Метод 3 — POST /api/games (добавить игру)

```
[HttpPost]
public ActionResult<Game> Create([FromBody] Game game) {
    game.Id = GamesStore.NextId();
    GamesStore.Games.Add(game);
    return CreatedAtAction(nameof(GetById), new { id = game.Id }, game);
}
```

[HttpPost] — этот метод срабатывает на POST-запрос по маршруту api/games.

[FromBody] Game game — параметр game берётся из тела запроса. Клиент присылает JSON, ASP.NET Core автоматически превращает его в объект Game. **[FromBody]** явно указывает: «ищи данные в теле запроса, а не в URL».

game.Id = GamesStore.NextId() — присваиваем игре новый уникальный Id. Мы не доверяем Id от клиента — сервер сам решает, какой Id присвоить.

GamesStore.Games.Add(game) — добавляем игру в список. **Add()** — стандартный метод списка **List<T>**.

return CreatedAtAction(...) — возвращает статус 201 Created (не 200, а именно 201 — значит «объект создан»). Принимает три аргумента:

- **nameof(GetById)** — имя метода, по которому можно получить созданный объект. **nameof** возвращает имя как строку, но безопаснее чем писать "GetById" вручную — при переименовании метода компилятор сам обновит эту ссылку
- **new { id = game.Id }** — параметры маршрута для того метода (то есть {id})
- **game** — сам созданный объект, который вернётся клиенту

4.6. Метод 4 — DELETE /api/games/{id} (удалить игру)

```
[HttpDelete("{id}")]
public ActionResult Delete(int id) {
    var game = GamesStore.Games.FirstOrDefault(g => g.Id == id);
    if (game is null) {
        return NotFound(new { message = $"Игра с id={id} не найдена" });
    }
    GamesStore.Games.Remove(game);
    return NoContent();
}
```

public ActionResult Delete — обратите внимание: здесь **ActionResult** без **<Game>**. Это потому, что при удалении мы не возвращаем никаких данных — только статус.

Поиск и проверка на null — те же, что в **GetById**, разбирали выше.

GamesStore.Games.Remove(game) — удаляем найденный объект из списка. **Remove()** — стандартный метод `List<T>`, принимает сам объект (не `Id`).

return NoContent() — возвращает статус 204 No Content. Это стандартный ответ на успешное удаление: «всё прошло хорошо, но возвращать нечего».

4.7. Метод 5 — PUT /api/games/{id} (обновить игру)

```
[HttpPut("{id}")]
public ActionResult<Game> Update(int id, [FromBody] Game updated) {
    var game = GamesStore.Games.FirstOrDefault(g => g.Id == id);
    if (game is null) {
        return NotFound(new { message = $"Игра с id={id} не найдена" });
    }
    game.Title = updated.Title;
    game.Genre = updated.Genre;
    game.ReleaseYear = updated.ReleaseYear;
    return Ok(game);
}
```

[HttpPut("{id}")] — срабатывает на PUT-запрос по маршруту `api/games/{id}`.

int id, [FromBody] Game updated — два параметра одновременно: `id` приходит из URL, `updated` — из тела запроса. ASP.NET Core разбирает их из разных мест автоматически.

game.Title = updated.Title и т.д. — мы не заменяем объект целиком, а обновляем поля по одному. Почему так, а не просто `game = updated`? Потому что объект `game` — это ссылка на элемент внутри списка `GamesStore.Games`. Если мы напишем `game = updated`, мы просто перепишем локальную переменную, а список не изменится. Поэтому мы явно меняем каждое поле. Представьте, что `game` — это стикер с адресом дома. Если написать `game = updated`, вы просто переклеили стикер на другой дом. Сам старый дом (объект в списке) не изменился. Чтобы изменить дом, нужно зайти внутрь и переставить мебель — то есть поменять поля `game.Title = updated.Title` и т.д.

Id не обновляем — это намеренно. `Id` — это идентификатор, он не должен меняться после создания.

return Ok(game) — возвращаем обновлённый объект с кодом 200.

Итоговая таблица методов

Метод	Маршрут	HTTP-статус	Что делает
GetAll	GET /api/games	200	Возвращает весь список
GetById	GET /api/games/{id}	200 / 404	Одну игру или ошибку
Create	POST /api/games	201	Добавляет новую игру
Delete	DELETE /api/games/{id}	204 / 404	Удаляет или ошибку
Update	PUT /api/games/{id}	200 / 404	Обновляет или ошибку

Шаг 5. Настройка Program.cs

Откройте **Program.cs** и полностью замените содержимое (вручную перепишите):

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

var app = builder.Build();

app.UseHttpsRedirection();
app.MapControllers();

app.Run();
```

- **var builder = WebApplication.CreateBuilder(args);** Создаём строителя приложения — он собирает все настройки и сервисы до запуска.
- **builder.Services.AddControllers();** Регистрируем поддержку контроллеров. Без этой строки ASP.NET Core не будет знать, что в проекте есть классы-контроллеры.
- **var app = builder.Build();** Завершаем настройку и создаём само приложение. После этой строки добавлять сервисы уже нельзя.
- **app.UseHttpsRedirection();** Если клиент обратился по http:// — автоматически перенаправляем его на https://.
- **app.MapControllers();** Подключаем маршруты из контроллеров. ASP.NET Core сканирует все классы с [ApiController] и регистрирует их маршруты. В ЛР №27 мы писали MapGet(...) вручную — теперь это происходит автоматически через атрибуты [HttpGet], [HttpPost] и т.д.

- **app.Run();** Запускаем сервер. Программа остаётся в этой строке и слушает входящие запросы.

Практическое задание

1. Запустите сервер:

cd GamesApi

dotnet run

2. Обратите внимание на вывод в терминале — там будет адрес сервера, например:



```
info: Microsoft.Hosting.Lifetime[14]  
Now listening on: http://localhost:5218
```

Запомните порт — он понадобится для curl.

3. Откройте браузер и проверьте GET-маршруты (вводите ваш порт):

http://localhost:5000/api/games

http://localhost:5000/api/games/1

http://localhost:5000/api/games/99

Третий адрес должен вернуть JSON с сообщением об ошибке и статусом 404.

4. Сделайте скриншот браузера с ответом /api/games и сохраните в **img/step5_browserLab28_FIO.png**.

Выполните в терминале:

git add .

git commit -m "Step 5: GamesController with GET, POST, DELETE, PUT"

git push

Шаг 6. Тестирование через curl

Откройте **новый терминал** в VS Code (старый оставьте — там работает сервер).

6.1. GET-запросы

Введите в терминале запросы (вводите ваш порт):

1. Получить все игры

curl http://localhost:5000/api/games

2. Получить игру с id=1

curl http://localhost:5000/api/games/1

3. Попробовать получить несуществующую игру

curl http://localhost:5000/api/games/99

Вы увидите JSON прямо в терминале. Для последнего запроса — сообщение об ошибке.

6.2. Добавление игры через POST

1. Введите в терминале (**вводите ваш порт, обратите внимание команды копируются не всегда корректно с PDF, вставьте их сперва в блокнот**):

```
curl -X POST http://localhost:5000/api/games \
  -H "Content-Type: application/json" \
  -d '{"title": "Stardew Valley", "genre": "Simulation", "releaseYear": 2016}'
```

Символ \ в конце строки означает, что команда продолжается на следующей строке — это одна команда, просто разбитая для читаемости

Сервер должен вернуть созданную игру с новым Id и статусом 201.

2. Проверьте, что игра добавилась:

```
curl http://localhost:5000/api/games
```

В списке теперь 4 игры.

6.3. Обновление игры через PUT

1. Обновить игру с id=2 (вводите ваш порт):

```
curl -X PUT http://localhost:5000/api/games/2 \
  -H "Content-Type: application/json" \
  -d '{"title": "Witcher 2: Assassins of Kings", "genre": "Action RPG", "releaseYear": 2011}'
```

2. Проверить что игра обновилась:

```
curl http://localhost:5000/api/games/2
```

3. Попробовать обновить несуществующую игру:

```
curl -X PUT http://localhost:5000/api/games/99 \
  -H "Content-Type: application/json" \
  -d '{"title": "Test", "genre": "Test", "releaseYear": 2020}'
```

Для первого запроса сервер вернёт обновлённую игру со статусом 200. Для третьего — сообщение об ошибке со статусом 404.

6.4. Удаление игры через DELETE

1. Удалить игру с id=1 (вводите ваш порт):

```
curl -X DELETE http://localhost:5000/api/games/1
```

2. Проверить что игра удалилась (вводите ваш порт):

```
curl http://localhost:5000/api/games
```

3. Попробовать удалить несуществующую игру (вводите ваш порт):

curl -X DELETE http://localhost:5000/api/games/99

4. Проверить через PUT:

**curl -X PUT http://localhost:5000/api/games/2 **

**-H "Content-Type: application/json" **

-d '{"title": "The Witcher 3", "genre": "RPG", "releaseYear": 2015}'

6.5. Посмотреть статусные коды

По умолчанию **curl** показывает только тело ответа. Чтобы увидеть статусный код, добавьте флаг **-i**:

curl -i http://localhost:5000/api/games/2

```
>$ curl -i http://localhost:5218/api/games/2
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Date: Mon, 02 Mar 2026 17:34:53 GMT
Server: Kestrel
Transfer-Encoding: chunked
```

curl -i -X DELETE http://localhost:5000/api/games/99

Вы увидите полный ответ включая статус:

```
>$ curl -i -X DELETE http://localhost:5218/api/games/99
HTTP/1.1 404 Not Found
Content-Type: application/json; charset=utf-8
Date: Mon, 02 Mar 2026 17:35:17 GMT
Server: Kestrel
Transfer-Encoding: chunked

{"message": "Игра с id=99 не найдена"}
```

Практическое задание

1. Выполните все команды **curl** из шагов 6.1–6.6
2. Добавьте через POST свою любимую игру
3. Убедитесь что она появилась в списке через GET
4. Сделайте скриншот терминала с результатом **curl -i** для несуществующей игры (статус 404)
5. Сохраните в **img/step6_curlLab28_FIO.png**

Выполните в терминале:

cd ..

git add .

git commit -m "Step 6: tested all endpoints with curl"

git push

cd GamesApi

Самостоятельное задание

Задание 1. Добавьте поле **IsFavourite**

Добавьте в модель **Game** новое поле:

```
public bool IsFavourite { get; set; } = false;
```

Добавьте маршрут **GET /api/games/favourites** который возвращает только игры где **IsFavourite = true**

Проверьте через **curl**: добавьте игру с **"isFavourite": true** и убедитесь что она появляется в **/api/games/favourites**

Подсказка: объявите маршрут **/favourites** в контроллере выше метода **GetById**, иначе ASP.NET Core попытается прочитать слово **"favourites"** как число и вернёт ошибку

Задание 2. Защита от пустого названия

В методе **Create** добавьте проверку: если **game.Title** пустое — возвращайте **BadRequest** со статусом **400** и сообщением «Название игры не может быть пустым».

Проверьте через **curl**:

```
curl -X POST http://localhost:5000/api/games \  
-H "Content-Type: application/json" \  
-d '{"title\":"\", \"genre\": \"RPG\", \"releaseYear\": 2020}'
```

Должен вернуться статус 400.

Задание 3. README.md

Создайте **README.md** в корне проекта со следующим содержимым:

1. Название и описание проекта
2. Инструкция по запуску (одна команда)
3. Таблица всех маршрутов:

Метод	Маршрут	Описание	Статус
GET	/api/games	Получить все игры	200
GET	/api/games/{id}	Получить игру по id	200 / 404
POST	/api/games	Добавить игру	201
DELETE	/api/games/{id}	Удалить игру	204 / 404

4. Примеры curl-команд для каждого маршрута

Выполните в терминале:

cd ..

git add .

git commit -m "Added complete README for Lab28 and tasks"

git push

Итоговая таблица: что изучили в лабораторной

Концепция	Описание
public static class	Класс существует в одном экземпляре, данные общие для всего приложения
List<Game>	Список объектов в памяти — простое хранилище без базы данных
FirstOrDefault(...)	Найти первый элемент по условию, вернуть null если не найден
[HttpPost]	Маршрут принимает POST-запросы
[HttpDelete("{id}")]	Маршрут принимает DELETE-запросы с параметром в URL
[FromBody]	Данные берутся из тела запроса (JSON)
Ok(...)	Вернуть статус 200 с данными
CreatedAtAction(...)	Вернуть статус 201 с адресом нового объекта
NotFound(...)	Вернуть статус 404 с сообщением
NoContent()	Вернуть статус 204 — успех без тела ответа
curl	Инструмент командной строки для отправки HTTP-запросов
curl -X POST	Указать метод запроса
curl -H "..."	Добавить заголовок к запросу
curl -d "..."	Тело запроса
curl -i	Показать статусный код и заголовки ответа